Quick Summary: Two Sum vs Two Sum II

Quick Comparison

Feature	Two Sum (LeetCode 1)	Two Sum II (LeetCode 167)
Input Array	**Unsorted**	**Sorted**
Optimal Approach	**Hash Map**	**Two Pointers**
Time Complexity	**O(n)**	**O(n)**
Space Complexity	**O(n)**	**O(1)**
Index Constraint	Must preserve original in...	1-indexed (or fixed position)

The Unsorted Problem (LeetCode 1)

- **Primary Tool: Hash Map**
- **Why Hash Map?** The input array is **unsorted**.
- **Why not Two Pointers?** Sorting the array first would cost **O(n log n)** time and **destroy original index positions**, breaking key problem requirements.
- **Trade-off:** Achieves optimal **O(n) time complexity** at the cost of **O(n) extra memory space** to store complement values.

The Sorted Problem (LeetCode 167)

- **Primary Tool: Two Pointers** (Left & Right)
- **Why Two Pointers?** The input array is **already sorted** and the problem strictly limits space to **O(1)**.
- **How it Works:**
  - Place pointers at the **start** and **end** of the array.
  - **Sum too large?** Decrement the right pointer to reduce the value.
  - **Sum too small?** Increment the left pointer to increase the value.
- **Trade-off:** Achieves optimal **O(n) time complexity** with **zero extra memory cost (O(1) space)**.

Bottom Line

- **Hash Map** is for **unsorted data** when you have **memory to spare** (O(n) space).
- **Two Pointers** is for **sorted data** when you need **O(1) auxiliary space**.